PYTHON REFERENCE GUIDE

Loops (Day 3) | Conditional Statements (Day 4)

This guide covers the second half of Day 3 (loops) and all of Day 4 (conditionals) from your roadmap — the pieces missing from your earlier Lists/Tuples/Dictionaries/Sets guide. Plain language, every concept has a working example and its output.

1	**For Loop**	Looping through lists, strings, and range()
2	**While Loop**	Looping based on a condition, break, continue
3	**If / Else**	Basic decision-making in Python
4	**Elif**	Checking multiple conditions in order
5	**Nested If**	An if statement inside another if statement
6	**Logical Operators**	and, or, not — combining conditions

1 FOR LOOP

A **for loop** repeats a block of code once for every item in a collection (a list, string, or range of numbers).

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

In plain English: Read it as: 'for each fruit in the fruits list, print that fruit.' The loop runs once per item, automatically.

Looping with range()

range() generates a sequence of numbers. Very common when you want to repeat something a fixed number of times.

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

Watch out: *range(5) gives 0,1,2,3,4 — five numbers, but stopping BEFORE 5. Same 'stop not included' rule as list slicing.*

range(start, stop, step)

```
for i in range(2, 10, 2):
    print(i)
```

Output:

```
2
4
6
8
```

Looping through a string

```
for letter in "cat":
    print(letter)
```

Output:

```
c
a
t
```

2 WHILE LOOP

A **while loop** keeps repeating **as long as a condition is True**. Unlike a for loop, it doesn't know in advance how many times it will run.

```
count = 1
while count <= 5:
    print(count)
    count = count + 1
```

Output:

```
1
2
3
4
5
```

Watch out: *If you forget to update count inside the loop, the condition never becomes False, and your program runs forever — this is called an 'infinite loop.' Always make sure something inside the loop moves it toward stopping.*

break — stop the loop early

Immediately exits the loop, even if the condition is still True.

```
count = 1
while count <= 10:
    if count == 4:
        break
    print(count)
    count += 1
```

Output:

```
1
2
3
```

continue — skip to the next round

Skips the rest of the current loop cycle and jumps straight to the next one, without stopping the whole loop.

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Output:

```
1
2
4
5
```

In plain English: break = 'leave the loop completely.' continue = 'skip just this one round, keep looping.'

3 IF / ELSE

An **if statement** runs a block of code only when a condition is True. **else** runs when it's False.

```
age = 20
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Output:

```
Adult
```

Watch out: *Python uses indentation (spaces) to know what belongs inside the if block — unlike some languages, there are no curly braces. Getting the indentation wrong causes errors.*

Comparison operators used in conditions

Operator	Meaning	Example
==	equal to	age == 18
!=	not equal to	age != 18
>	greater than	age > 18
<	less than	age < 18
>=	greater than or equal to	age >= 18
<=	less than or equal to	age <= 18

4 ELIF

elif (short for 'else if') lets you check **multiple conditions in order**. Python checks each one from top to bottom and stops at the first True match.

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Grade D")
```

Output:

```
Grade B
```

In plain English: Even though marks=75 also satisfies the 'marks >= 60' condition later, Python stops at the FIRST True condition it finds — elif never checks further once one matches.

You can chain as many elif as you need

```
day = 3
if day == 1:
    print("Monday")
elif day == 2:
    print("Tuesday")
elif day == 3:
    print("Wednesday")
else:
    print("Unknown day")
```

Output:

```
Wednesday
```

5 NESTED IF

A **nested if** is simply an if statement placed **inside** another if statement — used when a second decision only matters after the first condition is already True.

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("Need valid ID")
else:
    print("Too young")
```

Output:

```
Entry allowed
```

Watch out: *The inner if/else only runs at all if the outer condition (age >= 18) was True. If age was 15, Python would never even look at has_id.*

6 LOGICAL OPERATORS

Logical operators let you **combine multiple conditions** into a single if statement, instead of nesting.

and — both conditions must be True

```
age = 20
has_id = True
if age >= 18 and has_id:
    print("Entry allowed")
else:
    print("Not allowed")
```

Output:

```
Entry allowed
```

In plain English: This does the exact same job as the nested if example on the previous page, just in one line.

or — at least one condition must be True

```
is_weekend = False
is_holiday = True
if is_weekend or is_holiday:
    print("No work today")
else:
    print("Work day")
```

Output:

```
No work today
```

not — flips True to False, and False to True

```
is_raining = False
if not is_raining:
    print("Go for a walk")
```

Output:

```
Go for a walk
```

Quick summary table

Operator	True when...	Example
and	BOTH sides are True	age >= 18 and has_id
or	AT LEAST ONE side is True	is_weekend or is_holiday
not	the condition is False (flips it)	not is_raining